

CreditWorks Software Engineer – Technical Programming Assignment

Introduction

Thank you for your interest in the **Software Engineer** position at **CreditWorks**.

As part of our recruitment process, we would like you to complete a practical programming assignment.

The purpose of this exercise is not simply to determine whether the application works. We are interested in understanding how you:

- Analyse and interpret business requirements.
- Structure and organise a software solution.
- Model data and business rules.
- Design maintainable and testable application logic.
- Handle validation and edge cases.
- Make architectural and technical decisions.
- Write clear, readable and maintainable code.
- Test your implementation.
- Document assumptions and technical decisions.

You should approach the assignment as if you were implementing a small production application that may need to be maintained and extended by another developer in the future.

1. Assignment Overview

Create a web application for managing **vehicles and vehicle weight categories**.

Users must be able to:

1. Add vehicle details.
2. View all vehicles.
3. Sort the vehicle list.
4. Automatically determine a vehicle's category based on its weight.
5. Create, modify and delete vehicle categories.
6. Assign an icon to each category.

7. Maintain valid category ranges without gaps or overlaps.

8. See vehicle categories update automatically when category definitions change.

The application must persist its data in a relational database.

2. Vehicle Details

The system must allow a user to create a vehicle containing the following information:

- **Owner's Name**
- **Manufacturer**
- **Year of Manufacture**
- **Weight in kilograms**

All fields are mandatory.

Weight

Vehicle weight must:

- Be stored in kilograms.
- Support up to **two decimal places**.
- Be a positive value.
- Be used to determine the vehicle's current category.

For example:

1850.75 kg

3. Vehicle Manufacturers

Manufacturers are predefined.

The initial application must contain the following manufacturers:

1. Mazda
2. Mercedes
3. Honda
4. Ferrari

5. Toyota

Although the manufacturer list is expected to change rarely, the solution should be designed so that manufacturers are not unnecessarily embedded throughout the application.

We are interested in seeing how you model information that is currently fixed but may change in the future.

4. Vehicle Categories

Every vehicle must belong to exactly **one weight category**.

The category is determined automatically from the vehicle's weight.

The initial application must contain the following categories:

Category Weight Range

Light Up to 500 kg

Medium 500 kg to 2500 kg

Heavy 2500 kg and above

Each category must also have an associated **icon or graphic**.

You may choose any suitable icons.

5. Category Administration

Users must be able to:

- Create categories.
- Edit existing categories.
- Delete categories.
- Define the weight range for each category.
- Assign an icon to each category.

Category configuration is an important part of the assignment.

The application must enforce the following business rules.

5.1 Complete Weight Coverage

All valid vehicle weights must map to a category.

There must be **no gaps** between category ranges.

For example, the following configuration would be invalid:

- Light: 0–500 kg
- Medium: 600–2500 kg

because weights between 500 kg and 600 kg would not have a category.

5.2 No Category Overlap

Category ranges must not overlap.

Every possible vehicle weight must resolve to **exactly one category**.

For example, the following configuration would be invalid:

- Light: 0–600 kg
- Medium: 500–2500 kg

because a vehicle weighing 550 kg could belong to two categories.

5.3 Boundary Handling

Your solution should define clearly how category boundaries are handled.

For example, if one category ends at 500 kg and another begins at 500 kg, the system must ensure that a vehicle weighing exactly:

500.00 kg

belongs to one and only one category.

Document your chosen boundary rules in the README or development notes.

6. Category Changes and Existing Vehicles

Vehicle categories must reflect the **current category configuration**.

If category definitions change, the vehicle list must immediately display the correct new category for each existing vehicle.

For example, assume the original categories are:

- Light: up to 500 kg
- Medium: 500–2500 kg
- Heavy: 2500 kg and above

and a vehicle weighs:

2200 kg

The vehicle would initially be categorised as **Medium**.

If the categories are subsequently changed so that:

- Medium ends at 2000 kg
- Heavy begins at 2000 kg

the same vehicle must now appear as **Heavy**.

Your implementation should ensure that category information cannot become inconsistent with the configured category ranges.

7. Vehicle List

The application must provide a page displaying all vehicles stored in the database.

For each vehicle, display:

- Owner's Name
- Manufacturer
- Year of Manufacture
- Category icon

For example:

John Smith | Mazda | 2019 | [Light Category Icon]

Jane Turei | Mercedes | 2015 | [Medium Category Icon]

The underlying vehicle weight does not necessarily need to be shown, but it must be available to the application because it is used for categorisation and sorting.

8. Sorting

The vehicle list must support sorting by:

- Owner's Name
- Manufacturer
- Year of Manufacture
- Weight

The user should be able to clearly identify the currently selected sort order.

Both ascending and descending sorting is encouraged.

9. Validation

The application should perform appropriate validation.

At minimum, consider validation for:

Vehicle information

- Owner's Name is required.
- Manufacturer is required.
- Year of Manufacture is required and sensible.
- Weight is required.
- Weight must be positive.
- Weight supports a maximum of two decimal places.

Categories

- Category name is required.
- Category icon is required.
- Category ranges must be valid.
- Category ranges must not overlap.
- Category ranges must not contain gaps.
- Every valid vehicle weight must resolve to exactly one category.

Validation errors should be clearly communicated to the user.

The application should not depend exclusively on browser-side validation to maintain data integrity.

10. Persistence

Vehicle and category information must be persisted using:

Microsoft SQL Server 2019 or later

SQL Server Express is acceptable.

Your solution should contain everything required to create or initialise the database.

Examples include:

- Entity Framework migrations
- Database creation scripts
- Seed data
- Initialisation code

A reviewer should be able to clone the repository and establish the required database without having to manually reverse-engineer the schema.

11. Backend Technology

The original CreditWorks assignment specifies C# middleware and Microsoft SQL Server.

For this Software Engineer assignment, implement the backend using:

- **C#**
- **ASP.NET Core**
- **Entity Framework Core** or another clearly justified .NET data-access approach
- **Microsoft SQL Server 2019 or later**

Use a currently supported **.NET version** unless CreditWorks provides a specific version before the test is issued.

Your solution should demonstrate an appropriate separation between:

- Presentation/API concerns
- Business rules
- Data access
- Domain or application logic

We are not prescribing a particular architecture. Use an architecture appropriate to the size of the application and be prepared to explain your choices.

12. Frontend Technology

You may use an appropriate frontend technology of your choice.

Examples include:

- Blazor
- ASP.NET Core MVC / Razor Pages
- Angular
- React
- Another appropriate web framework

The frontend technology itself is not the primary focus of the exercise.

We are more interested in whether the resulting application is:

- Functional
- Easy to understand
- Responsive to user actions
- Appropriately validated
- Structured sensibly
- Maintainable

A polished visual design is welcome but is not expected to take priority over software quality.

13. Automated Testing

Include automated tests covering important application behaviour.

We are particularly interested in tests around business rules, for example:

- Correct category determination.
- Category boundary values.
- Prevention of overlapping category ranges.

- Prevention of gaps between ranges.
- Category changes affecting existing vehicles correctly.
- Vehicle validation.
- Sorting where appropriate.

You do not need to test every trivial implementation detail.

Focus testing on behaviour where defects would have meaningful consequences.

Where appropriate, separate unit tests from integration tests.

14. Error Handling

The application should handle expected errors appropriately.

Examples include:

- Invalid user input.
- Invalid category configuration.
- Database errors.
- Attempts to delete or modify categories in ways that would leave an invalid configuration.
- Requests for records that do not exist.

Avoid exposing unnecessary technical implementation details or stack traces to the end user.

15. Code Quality

We will consider the general engineering quality of the submission.

This includes areas such as:

- Clear naming.
- Appropriate separation of responsibilities.
- Readable code.
- Reasonable use of abstractions.
- Avoidance of unnecessary duplication.

- Appropriate asynchronous programming.
- Correct resource handling.
- Sensible dependency management.
- Maintainable database design.
- Appropriate use of configuration.
- Useful automated tests.
- Clear handling of business rules.

We are not looking for unnecessary complexity or an excessive number of architectural layers.

Prefer a solution that is simple, clear and maintainable.

16. Security Considerations

Although this is a small programming exercise, normal secure-development practices should still be followed.

Consider areas such as:

- Server-side validation.
- SQL injection prevention.
- Safe handling of user input.
- Appropriate error responses.
- Configuration and connection strings.
- Avoiding secrets committed to source control.

Authentication and authorisation are **not required** unless you choose to implement them.

If you deliberately omit something that would normally be required in production, briefly note it in your development notes.

17. Database Design

You are free to design the database schema.

Your design should support the current requirements while allowing reasonable future changes.

We will be interested in decisions such as:

- How manufacturers are represented.
- How vehicle categories are represented.
- How category ranges are stored.
- Whether a vehicle's category is stored or calculated.
- How category icons are represented.
- How data integrity is maintained when category definitions change.

Document any important decisions or trade-offs.

18. README / Development Notes

Include a README containing enough information for another developer to run and understand your solution.

At minimum, include:

Setup

- Required software.
- Database requirements.
- Configuration steps.
- Database creation or migration instructions.
- How to build the application.
- How to run the application.
- How to execute automated tests.

Design Notes

Briefly explain:

- Overall solution architecture.
- Database design.
- Category calculation approach.

- Category boundary rules.
- Important assumptions.
- Significant design decisions.
- Known limitations.

You may also describe what you would improve if this application were developed further as a production system.

19. Source Control

Submit the application using a Git repository.

Your repository should contain:

- Complete application source code.
- Solution and project files.
- Automated tests.
- Database migrations or scripts.
- README.
- Development/design notes where applicable.
- Any assets required by the application.

Do not include:

- Compiled binaries.
- Build output.
- Database backup files unless specifically required.
- Passwords.
- API keys.
- Production connection strings.
- Other secrets.

A clear and sensible Git history is welcome but is not mandatory.

20. Running the Application

The reviewer should be able to:

1. Clone the repository.
2. Read the README.
3. Configure the database.
4. Apply migrations or initialise the database.
5. Build the solution.
6. Run the application.
7. Run the automated tests.

Avoid requiring undocumented manual steps.

21. What We Will Review

We will assess the submission across several areas.

Functional correctness

Does the application satisfy the stated requirements?

Software design

Is the solution structured appropriately and can the important design decisions be understood?

Business-rule implementation

Are category ranges, boundary conditions and category changes handled correctly?

Data design

Does the database model appropriately support the application?

Code quality

Is the code readable, maintainable and reasonably organised?

Testing

Are important behaviours protected by meaningful automated tests?

User experience

Is the application understandable and usable?

Documentation

Can another developer clone, configure, run and understand the solution?

Engineering judgement

Have appropriate trade-offs been made without unnecessarily over-engineering the solution?

22. What We Are Not Looking For

This assignment is not intended to measure how much code you can produce.

You do **not** need to:

- Build a large enterprise architecture.
- Introduce microservices.
- Implement authentication unless you choose to.
- Create production cloud infrastructure.
- Create an elaborate graphical design.
- Implement features unrelated to the requirements.

Additional functionality will not compensate for missing or incorrect core requirements.

Focus on producing a **complete, understandable and well-engineered solution**.

23. Assumptions and Questions

Software requirements are not always perfect.

If you identify ambiguity in the assignment:

1. Make a reasonable engineering decision.
2. Document your assumption.
3. Continue with the implementation.

You may also contact CreditWorks if you believe clarification is necessary.

Being able to recognise and clearly explain ambiguity is part of software engineering.

24. Submission

When your solution is complete, provide CreditWorks with the URL of the Git repository containing your submission.

The repository must contain all files necessary for us to review, build and run the application, together with your README and any relevant design or development notes.

The existing CreditWorks test asks candidates to provide a Git repository containing the solution, documentation, README and development notes.

25. Discussion at Interview

If you progress to the next stage, we may discuss your solution with you.

You should be prepared to explain:

- Your architecture.
- Your database model.
- How category assignment works.
- How you prevent gaps and overlaps.
- How you handle category boundary values.
- Your automated testing strategy.
- Any assumptions you made.
- Trade-offs you considered.
- What you would change if the application had to support significantly more users or data.
- What you would improve if you had additional development time.

We may also ask you to discuss how you would extend the application in response to a new business requirement.

The objective is to understand your engineering reasoning rather than whether you selected one particular architecture or framework.

Thank you for taking the time to complete the CreditWorks Software Engineer programming assignment.